

STAT 240

Week 8: Classification

Owen G. Ward

Today

- Midterm Review
- K-Nearest Neighbours
- Classification

Midterm Review

Some Common Issues, Comments

- Using `#| eval: false`
 - This needs to be the very first line in a code chunk!
 - The spacing matters, otherwise it won't be recognized
- We also saw in class how to just comment out everything at once at the start of the document
- If you get an error, it will run the code down to the error, then tell you then chunk it got stuck at
 - Only need to comment from there down!

More Comments

- Please don't print out all the data, it makes it harder for the TAs to grade
- Use `nrow`, `ncol` or `dim` to give the number of rows or columns
- Database is locked
 - This is probably because you didn't close the connection!

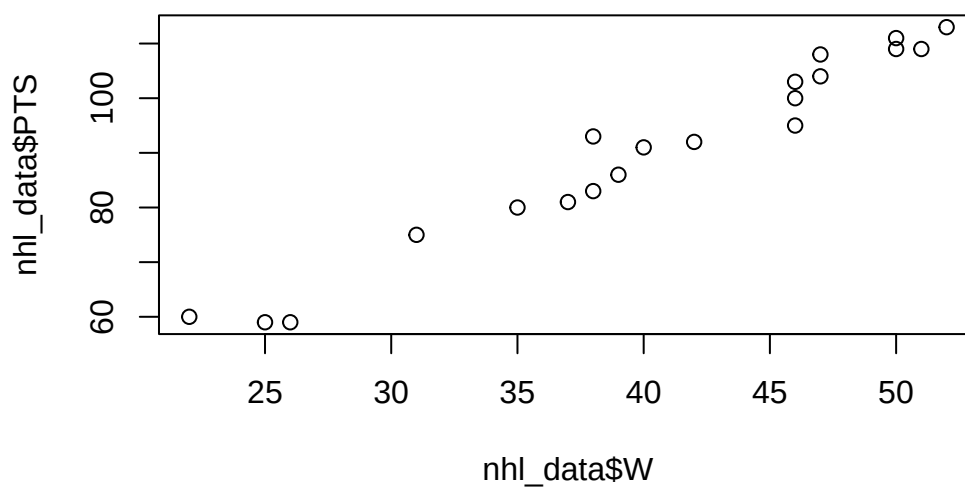
-
- Please keep your lines of code shorter, so that they don't run off the page
 - You can add new lines, even in SQL commands and it won't change the code

```
query <- sqlInterpolate(db, "SELECT SUM(Quantity) FROM sales
                             WHERE StockCode = '71053' AND
                             InvoiceDate LIKE ?month",
  month = paste0(months[i], "%")
)
```

Solutions are posted

- Will just go through the solution for Q3

```
plot(x = nhl_data$W, y = nhl_data$PTS)
```



Q3b

```
fit <- lm(PTS ~ W, data = nhl_data)
intercept <- fit$coefficients[1]
intercept
```

```
(Intercept)
  13.47326
```

```
slope <- fit$coefficients[2]
slope
```

```
W
1.90784
```

Q3c

```
SSE <- function(x, y, alpha, beta) {
  error <- y - alpha - beta * x
  sum(error^2)
}
```

```
SSE(x = nhl_data$W, y = nhl_data$PTS, alpha = intercept, beta = slope)
```

```
[1] 198.6633
```

```
SSE(x = nhl_data$W, y = nhl_data$PTS, alpha = 2, beta = 2)
```

```
[1] 1413
```

-
- Take a look through the solutions, let me know if any questions or confusion
 - Office Hours after class
 - Final coding exam will be very similar in format, make sure you know how to render the document, submit it!

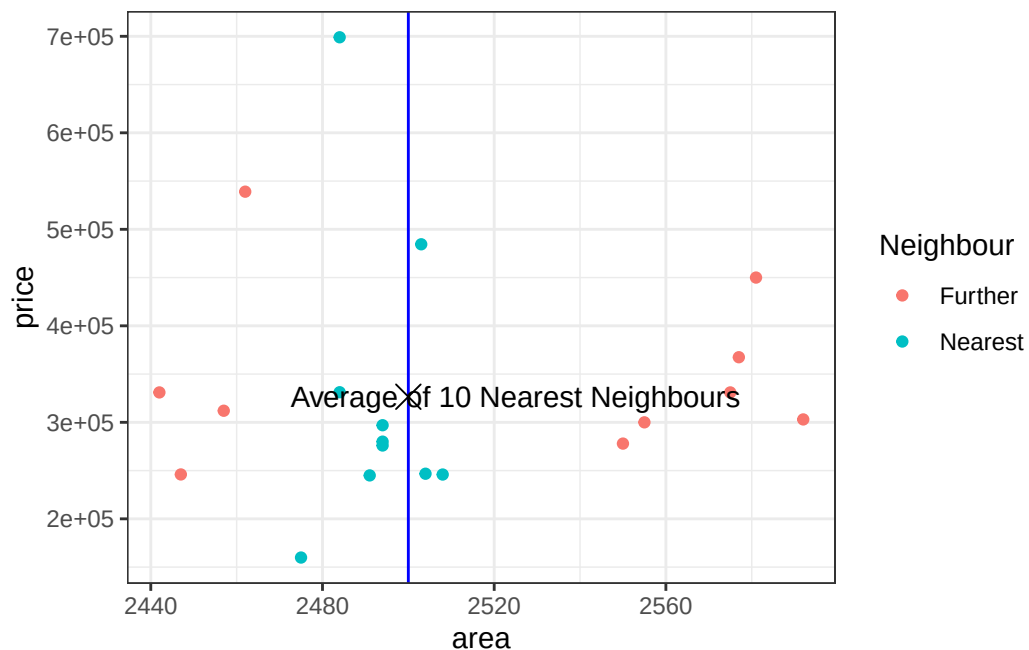
Quick Logistics

- Second coding exam April 10th
 - I'll keep some time April 3rd/8th to review for this
- Final written exam April 14th, 9.30am (1.5 hours)
- I will finish material April 3rd, review for written exam April 8th
 - You'll get some practice problems in advance

Recap: KNN

Last Time

```
house_sales <- read.csv("house_prices.csv")
```



What is KNN

- Pick a number of neighbours, K
- Our prediction for a given X is just the average Y of the K points nearest to that
- This is a very different approach to linear regression

Linear Regression

- Suppose a model for the data

$$Y = f(X) = \alpha + \beta X$$

- Estimate this model (estimate α, β) using all data in our training set
- Use this for predictions for all new values of X

Training and Test Data

```
set.seed(240)
prop_train <- 0.7
prop_test <- 1 - prop_train

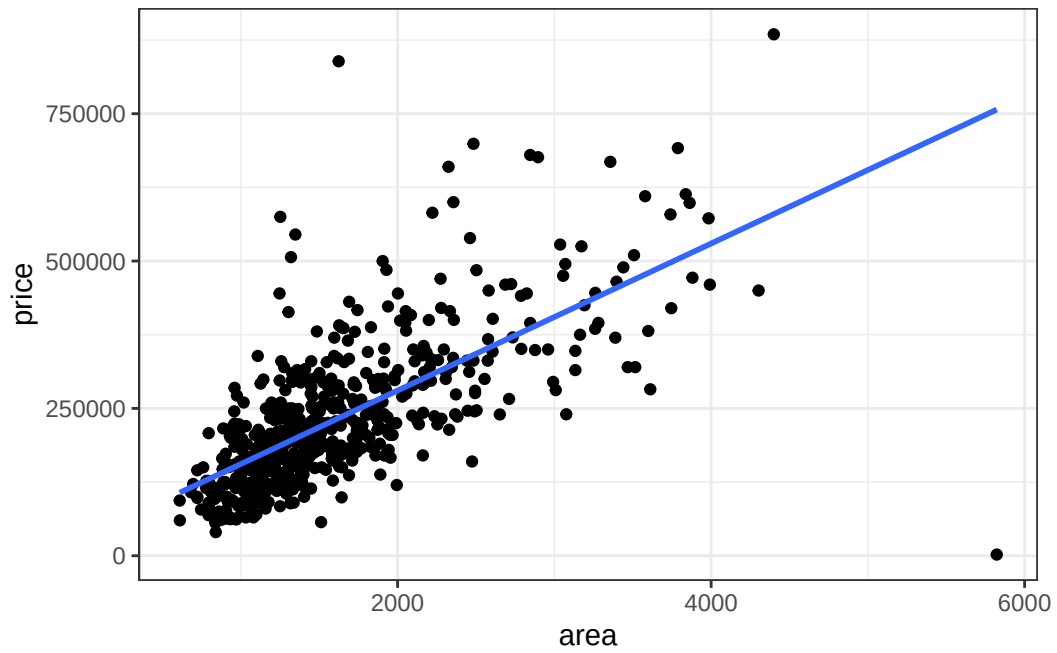
n_data <- nrow(house_sales)

train_test <- sample(c("Train", "Test"),
                    size = n_data, replace = TRUE,
                    prob = c(prop_train, prop_test))

train_data <- house_sales[train_test == "Train", ]
test_data <- house_sales[train_test == "Test", ]
```

Regression Predictions

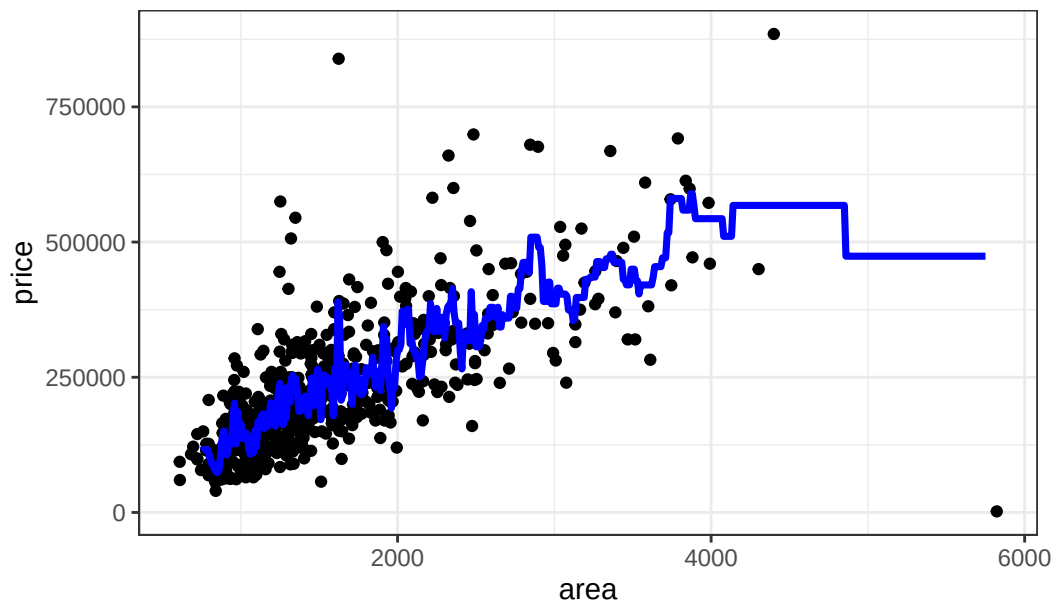
- Prediction for any X is $\hat{\alpha} + \hat{\beta}X$, value of Y on the line



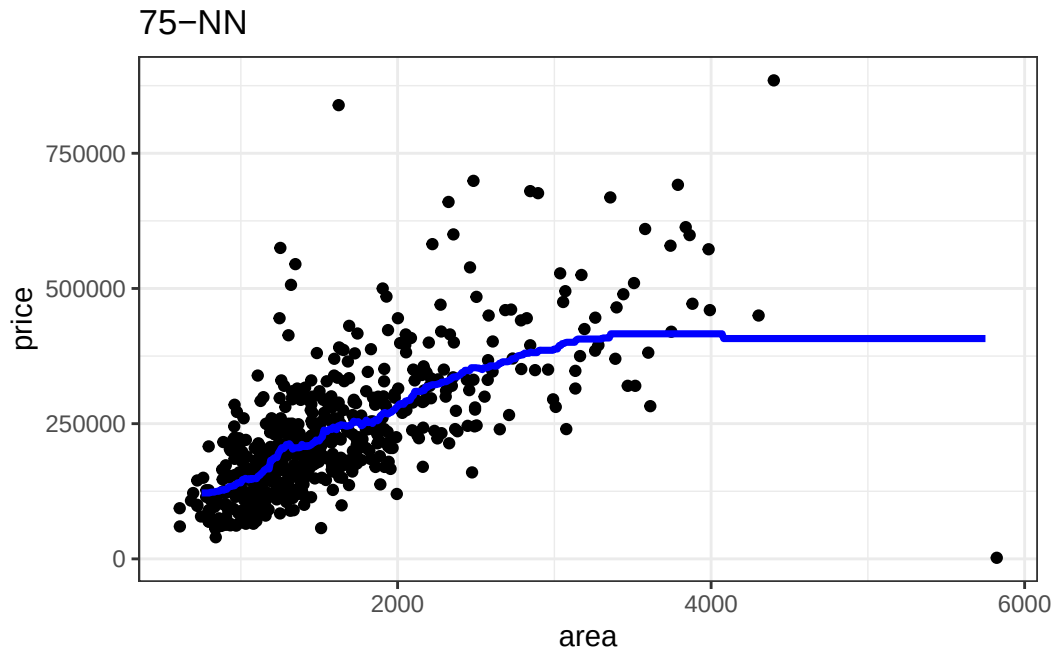
KNN Predictions

- KNN prediction the average of the K nearest points

5-NN



Different K



Differences between them

- KNN can give much more flexible predictions than regression
- As we saw for this data, doesn't necessarily mean it's a better prediction model
- We still haven't talked about how to choose the right K for KNN, but we know about training and test data and RMSPE, which we need

Choosing K

How to choose the best K

- We've seen how we can compare two prediction models by:
 - Fitting them both on a training set
 - Getting predictions for a test set
 - Seeing which gives the smallest RMSPE for the test set
- The approach for choosing K is quite similar

The key challenge

- When we use training and test data, we cannot use the test data to decide on the correct version of a specific model (the choice of K in KNN)
- We can only use the training data for this, otherwise we will overfit our model
 - Could end up with a model which doesn't actually give the best predictions for new data
- If your goal is getting the most accurate predictions (it is), this is a key thing to remember

How to deal with this

- So we want a way to compare different choices of K using only the training data
- What we do is to split the training data into another training and test set, using **Cross Validation**

-
- This step (choosing K) is known as **tuning** the prediction model
 - Has to be done in most prediction models where there are parameters which you get to choose
 - Linear Regression doesn't have this in the simple model we saw, but could tune which predictors to include in your model (stat 302/305)

Cross Validation

- Want to evaluate how good our model is for different choices of K , using only the training data
- Will just see a specific method, known as 5-fold Cross Validation
 - Break the training data into 5 equally sized groups
- Can do this for other choices than 5, but just going to use this here

5-fold Cross Validation

- Randomly split the training data into 5 equally sized pieces, V_1, V_2, V_3, V_4, V_5
- For a given choice of K in KNN:
 - Fit KNN to V_2, V_3, V_4, V_5 as training data
 - * Get RMSPE on V_1 as test data
 - Repeat with V_1, V_3, V_4, V_5 as training, RMSPE on V_2
 - Repeat with V_1, V_2, V_4, V_5 as training, RMSPE on V_3
 - Repeat with V_1, V_2, V_3, V_5 as training, RMSPE on V_4
 - Repeat with V_1, V_2, V_3, V_4 as training, RMSPE on V_5

Cross Validation

- The previous slide gives us:
 - Predictions for each of V_1, V_2, V_3, V_4, V_5 where each was test data and everything else used to train the model
- This is a way to treat all of the data in the training set like test data
- If we average these RMSPE's this is a measure of how good the model is, using only the training data!
 - But without overfitting

A picture of this

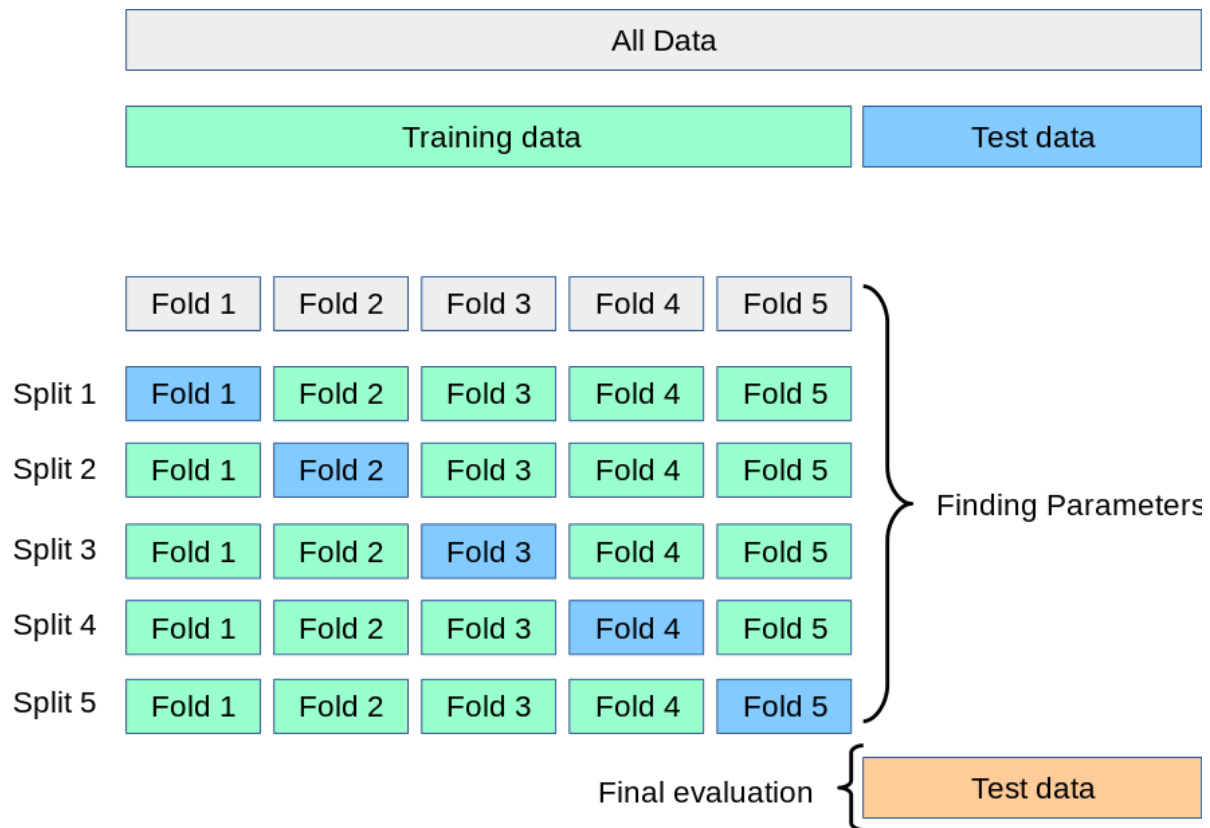


Figure 1: 5 Fold Cross Validation

Doing this for KNN

Compute this for 5-NN

```
set.seed(240)
CV_folds <- sample(1:5, size = nrow(train_data), replace = TRUE)
num_folds <- 5
CV_RMSPE <- rep(NA, num_folds)

for(i in 1:num_folds){
  cv_train <- train_data[CV_folds != i, ]
  cv_val <- train_data[CV_folds == i, ]
  fit_knn <- knnreg(price ~ area, data = cv_train, k = 5)
  cv_pred <- predict(fit_knn, cv_val)
```

```

  CV_RMSPE[i] <- sqrt( mean( (cv_pred - cv_val$price)^2 ) )
}

overall_RMSPE <- mean(CV_RMSPE)
overall_RMSPE

```

```
[1] 94089.11
```

Example

- Repeat this process for $K = 10$ nearest neighbours

Solution

Tuning this model

- What we have now is a way to compute the RMSPE on training data for any value of K
- Want to try a range of values of K , see which gives the best predictions
- Quite easy to do this given the code we have

Implementing this

```

K_vals <- seq(from = 1, to = 125, by = 1)
K_RMSPE <- rep(NA, length(K_vals))
set.seed(100)

CV_folds <- sample(1:5, size = nrow(train_data), replace = TRUE)
num_folds <- 5

for(j in 1:length(K_vals)) {
  k <- K_vals[j]
  CV_RMSPE <- rep(NA, num_folds)

  for(i in 1:num_folds){
    cv_train <- train_data[CV_folds != i, ]
    cv_val <- train_data[CV_folds == i, ]
    fit_knn <- knnreg(price ~ area, data = cv_train, k = k)
  }
}

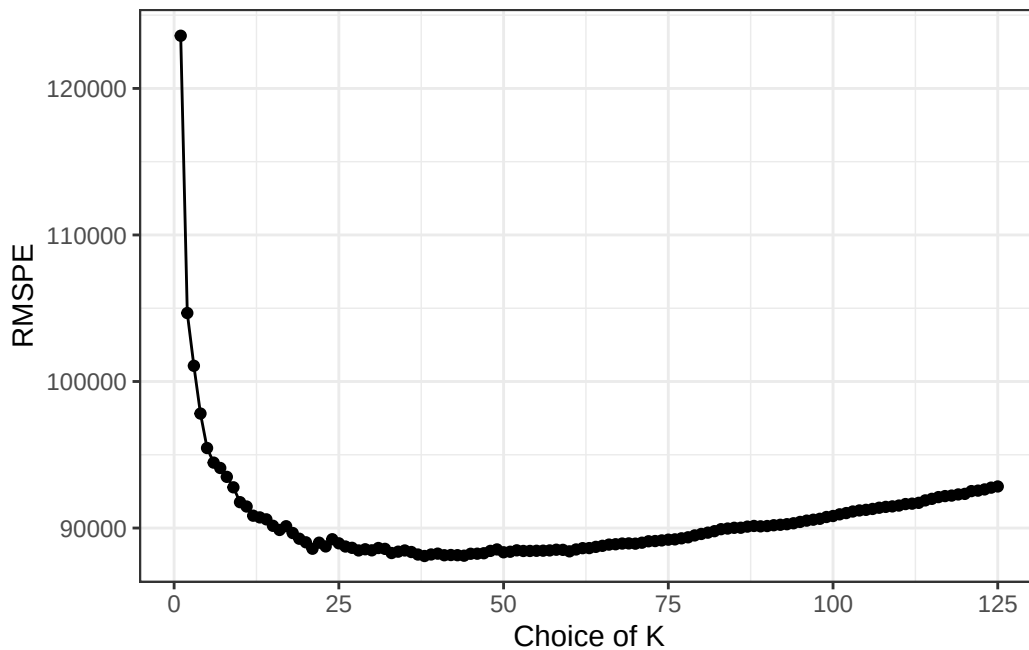
```

```

    cv_pred <- predict(fit_knn, cv_val)
    CV_RMSPE[i] <- sqrt( mean( (cv_pred - cv_val$price)^2 ) )
  }
  K_RMSPE[j] <- mean(CV_RMSPE)
}

```

Plotting this



Choosing the Best K

- All the values around 30-60 give very similar performance
- Can find the value of K which is best

```

K_vals[which.min(K_RMSPE)]

```

```

[1] 38

```

Comparing to Linear Regression

- So we've decided that $K = 38$ is the best possible KNN model
- We need to go back and compare that to linear regression on the test data
- To do this we refit to **all training data** using this best tuned model, get predictions on the test set

Comparing them

```
fit_train <- lm(price ~ area, data = train_data)
est_Y <- predict(fit_train, newdata = test_data)
RMSPE <- sqrt( mean( (est_Y - test_data$price) ^2) )
RMSPE
```

```
[1] 76317.91
```

```
fit_knn <- knnreg(price ~ area, data = train_data, k = 38)
est_Y_knn <- predict(fit_knn, test_data)
RMSPE_knn <- sqrt( mean( (est_Y_knn - test_data$price) ^2) )
RMSPE_knn
```

```
[1] 78744.96
```

- Still doesn't outperform linear regression for this problem!

What we've seen so far

- Two methods for doing prediction when we want to predict a numeric quantity
 - Linear Regression
 - K-Nearest Neighbours
- A way to compare these models with training and test data
- A way to tune the KNN model using only the training data (cross validation)

Next Steps

- What about if you want to predict something different, which is not a number?
- Will this house sell at this price or not?
- Should we approve Alice for a loan?
- The methods so far need to be modified to these settings!

Classification

Types of Data

- If Y is a number (number of units sold, price) we will call it a **numeric** variable
 - What we had before
- If Y can take on one of several possible categories (e.g., Red, White or Blue) we call it a **categorical** variable
- If Y can take on only **two** possible categories then we will call it a **binary** variable

Binary Data

- This is the other type of data we will consider now
- Binary because there are only two possible values: Yes/No, True/False, etc
- It is common to code this in data as 1/0, but we can't treat it as numeric
 - Be careful of this!

Prediction

- We can do prediction for binary data, we need a different approach to numeric data
- We know that Y can only take on 1 of 2 possible categories, so predicting some number doesn't really make sense
- We want some method which will tell us which category is more likely

Example

- We're going to take MLB data, pitches thrown by Shohei Ohtani in 2023
- We would like to identify which type of pitch he throws based on the location of the ball when he throws it
- We will only consider 2 types of pitches, Fastball (FF) or Sweeper (ST)

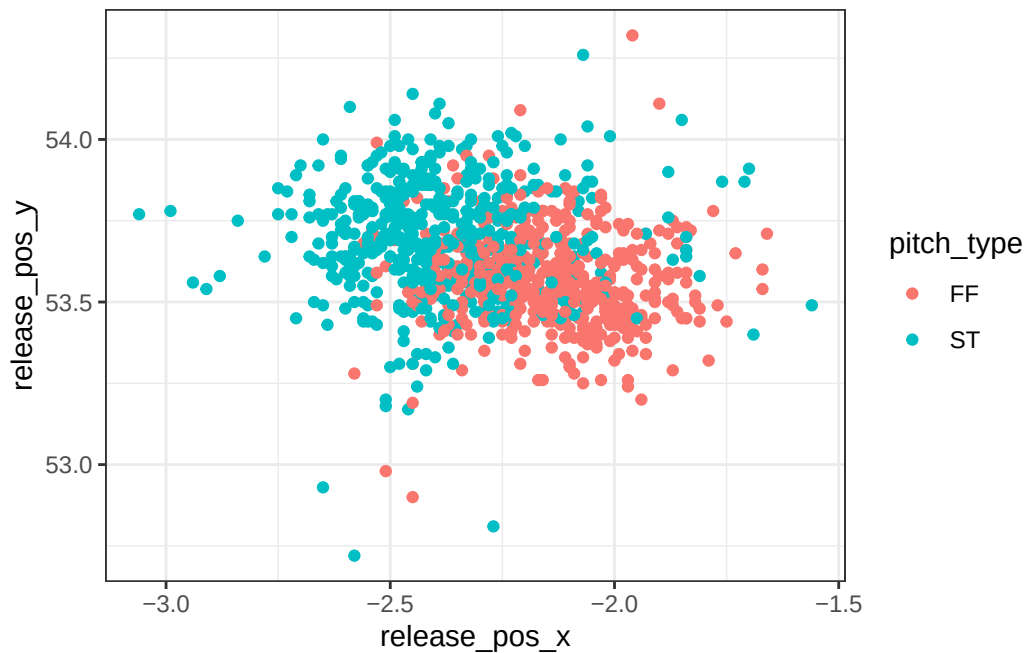
Getting the Data

- Access this data using an R package which queries an MLB API (no key needed). Data got updated, so need to install the package from Github!
- This code is a little slow to run, I will post the data on Canvas also

```
remotes::install_github("BillPetti/baseballr")
library(baseballr)
shohei_id <- 660271
shohei_data <- statcast_search_pitchers(start_date = "2023-03-01",
                                       end_date = "2023-12-01",
                                       pitcherid = shohei_id)

pitch_data <- shohei_data |>
  filter(pitch_type %in% c("FF", "ST")) |>
  select(pitch_type, release_speed, release_pos_x, release_pos_y,
         release_pos_z)
```

The Data



The Goal

- If we know the position of the pitch (location on the previous plot) can we predict which type of pitch it is?
- If we have some method which can predict this, how do we decide if it's any good?

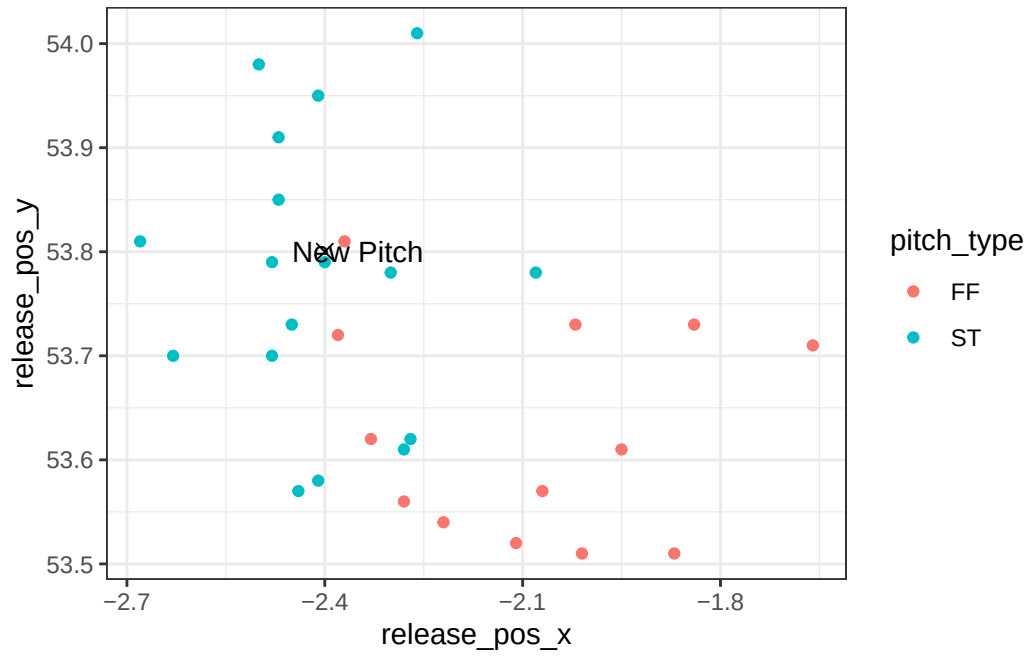
Linear Regression won't work

- Linear regression can only return a numeric value for Y , but here we want to predict one of two possible categories (Fastball/Sweeper)
- No way of ordering these categories either, can't convert them into numbers

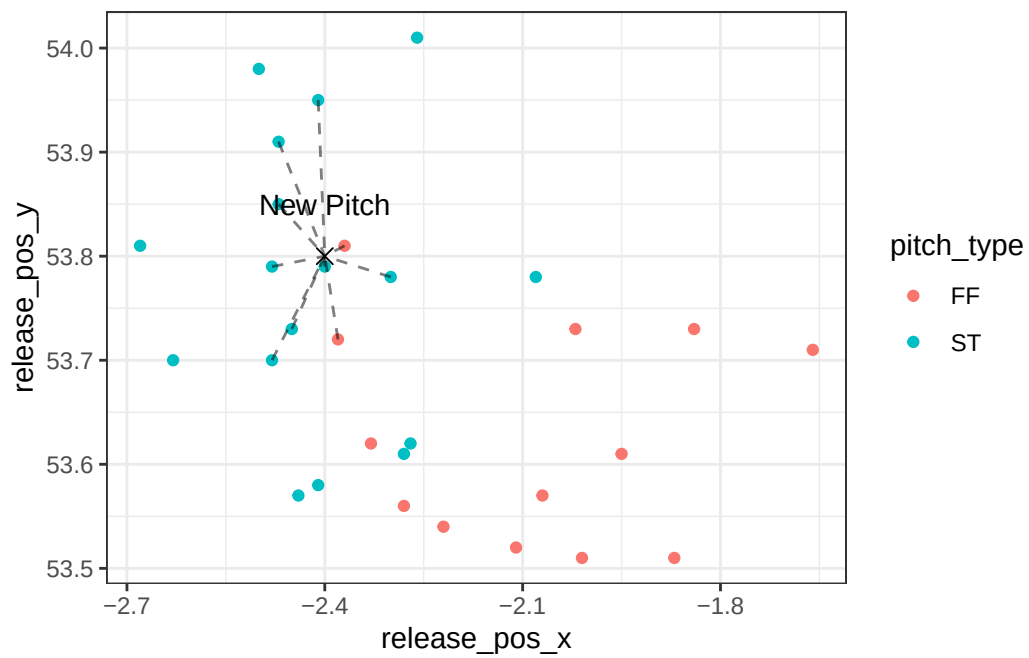
What about KNN?

- We can modify K-nearest neighbours to this problem
- Count how many of the K nearest points are in a given class
- Predict whichever class occurs more often

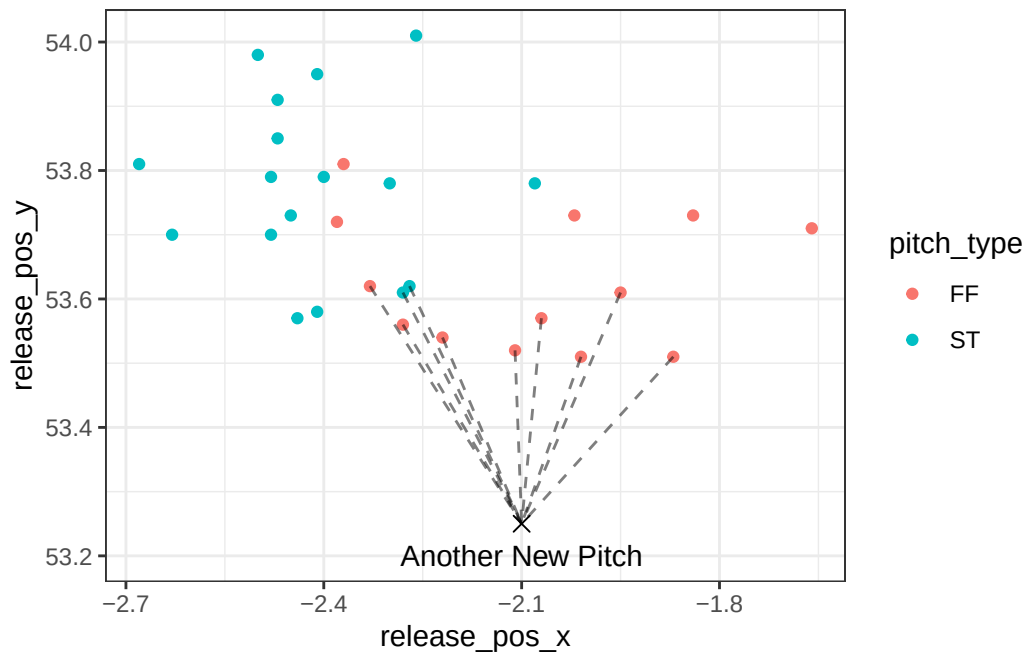
Find the nearest points where we want to predict



Getting Nearest Points



Predicting Somewhere Else



Computing the distance here

- We can compute this for any possible new pitch, once we know the x and the y
- To find the nearest points in the data, compute Euclidean distance

$$\text{distance} = \sqrt{(x - x_{\text{new}})^2 + (y - y_{\text{new}})^2}$$

- We can do this in higher dimensions too

How do we evaluate KNN here?

- We know how KNN makes predictions for this data
 - How do we pick the best model?
- RMSPE doesn't make sense, only works for numeric predictions
- KNN will give us a predicted class for any point, either Fastball or Sweeper (for this example)

Evaluating Models

- Count how many points it predicts the correct class for

$$\text{accuracy} = \frac{\text{how many correct predictions}}{\text{how many data points we predict at}}$$

- This is a number between 0 and 1
- Want values close to 1!

Fitting KNN to this data

```
pitch_data <- read.csv("pitch_data.csv")
set.seed(100)
prop_train <- 0.7
prop_test <- 1 - prop_train

n_data <- nrow(pitch_data)

train_test <- sample(c("Train", "Test"),
                     size = n_data, replace = TRUE,
                     prob = c(prop_train, prop_test))

training_data <- pitch_data[train_test == "Train", ]
testing_data <- pitch_data[train_test == "Test", ]
```

Predicting for Test Data

```
library(class)
fit <- knn(train = as.matrix(cbind(training_data$release_pos_x,
                                   training_data$release_pos_y)),
           test = as.matrix(cbind(testing_data$release_pos_x,
                                   testing_data$release_pos_y)),
           cl = training_data$pitch_type, k = 5)

table(fit, testing_data$pitch_type)
```

```
fit  FF  ST
FF 151  49
ST  60 189
```

- So there are 47 points where we predict FF and the true class is ST, etc
- What is the accuracy of this result? Compute it!

Solution

Computing the accuracy using R:

Exercise

Modify the CV code from earlier to find the best choice of K between $K = 1$ and $K = 200$ for this data. Which value of K gives the largest accuracy with CV?

- Hint:
 - Need to change RMSPE to accuracy!
 - Need to modify the KNN function to what we had on previous slides

Solution

Plotting these models

```
plot(K_vals, K_acc, type = "l", ylab = "Accuracy", xlab = "K")
which.max(K_acc)
```

Summary

- This has been a very quick introduction to classification, one simple classification method (KNN)
- You will study this a lot more in future classes (Stat 452)
- Will cover this a little more in the lab next week